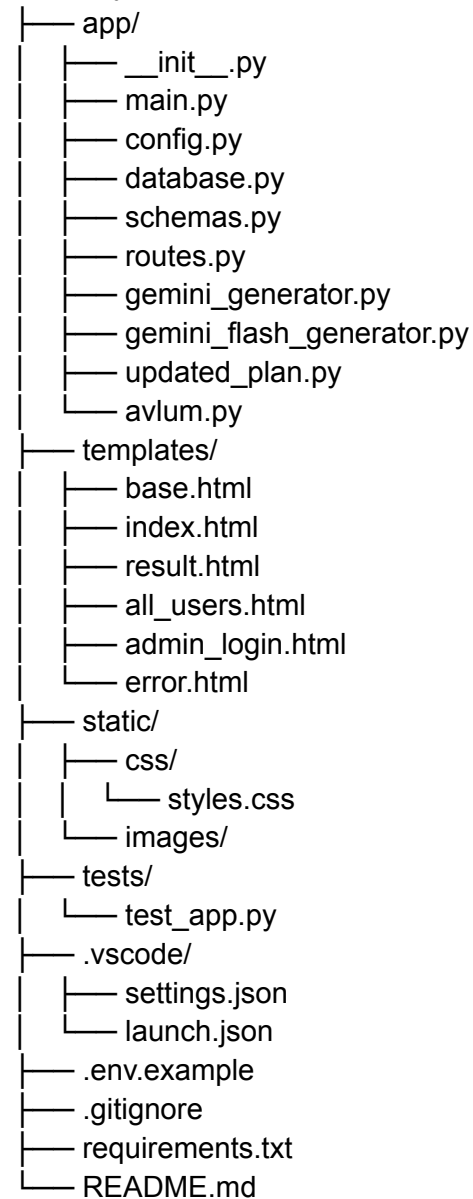


1. Project Structure

FitBuddy/



2. app/__init__.py

```
"""
```

FitBuddy application package.

```
"""
```

3. app/config.py

```
from pathlib import Path
```

```
from pydantic_settings import BaseSettings, SettingsConfigDict
```

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

```

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
    )

    app_name: str = "FitBuddy"
    debug: bool = True

    database_url: str = (
        f"sqlite:///{(BASE_DIR / 'fitbuddy.db').as_posix()}"
    )

    gemini_api_key: str = ""
    gemini_workout_model: str = "gemini-3.8-flash"
    gemini_nutrition_model: str = "gemini-3.8-flash"

    admin_password: str = "change-me"
    session_secret: str = "change-this-secret"

    max_feedback_chars: int = 1500

    avlum_enabled: bool = False
    avlum_webhook_url: str = ""
    avlum_api_key: str = ""

```

```

settings = Settings()
4. app/database.py
from datetime import datetime
from typing import Generator

```

```

from sqlalchemy import (
    DateTime,
    Float,
    Integer,
    String,
    Text,
    create_engine,
    select,
)
from sqlalchemy.orm import (

```

```

    DeclarativeBase,
    Mapped,
    Session,
    mapped_column,
    sessionmaker,
)

from .config import settings

connect_args = {}

if settings.database_url.startswith("sqlite"):
    connect_args = {
        "check_same_thread": False
    }

engine = create_engine(
    settings.database_url,
    connect_args=connect_args,
)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    autocommit=False,
    expire_on_commit=False,
)

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(
        Integer,
        primary_key=True,
        index=True
    )

```

```
user_id: Mapped[str] = mapped_column(
    String(80),
    unique=True,
    index=True
)
```

```
username: Mapped[str] = mapped_column(
    String(120)
)
```

```
age: Mapped[int] = mapped_column(
    Integer
)
```

```
weight: Mapped[float] = mapped_column(
    Float
)
```

```
goal: Mapped[str] = mapped_column(
    String(50)
)
```

```
intensity: Mapped[str] = mapped_column(
    String(20)
)
```

```
created_at: Mapped[datetime] = mapped_column(
    DateTime,
    default=datetime.utcnow
)
```

```
class Plan(Base):
```

```
    __tablename__ = "plans"
```

```
    id: Mapped[int] = mapped_column(
        Integer,
        primary_key=True,
        index=True
    )
```

```
    user_id: Mapped[str] = mapped_column(
        String(80),
```

```

        index=True
    )

    original_plan: Mapped[str] = mapped_column(
        Text
    )

    updated_plan: Mapped[str | None] = mapped_column(
        Text,
        nullable=True
    )

    nutrition_tip: Mapped[str] = mapped_column(
        Text
    )

    feedback: Mapped[str | None] = mapped_column(
        Text,
        nullable=True
    )

    created_at: Mapped[datetime] = mapped_column(
        DateTime,
        default=datetime.utcnow
    )

    updated_at: Mapped[datetime] = mapped_column(
        DateTime,
        default=datetime.utcnow,
        onupdate=datetime.utcnow
    )

def init_db() -> None:
    Base.metadata.create_all(
        bind=engine
    )

def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()

    try:
        yield db

```

```
finally:  
    db.close()
```

```
def save_user(  
    db: Session,  
    *,  
    user_id: str,  
    username: str,  
    age: int,  
    weight: float,  
    goal: str,  
    intensity: str,  
) -> User:
```

```
    existing = db.scalar(  
        select(User).where(  
            User.user_id == user_id  
        )  
    )
```

```
    if existing:  
        existing.username = username  
        existing.age = age  
        existing.weight = weight  
        existing.goal = goal  
        existing.intensity = intensity
```

```
        db.commit()  
        db.refresh(existing)
```

```
    return existing
```

```
    user = User(  
        user_id=user_id,  
        username=username,  
        age=age,  
        weight=weight,  
        goal=goal,  
        intensity=intensity,  
    )
```

```
    db.add(user)  
    db.commit()
```

```
db.refresh(user)
```

```
return user
```

```
def save_plan(  
    db: Session,  
    *,  
    user_id: str,  
    original_plan: str,  
    nutrition_tip: str,  
) -> Plan:
```

```
    plan = Plan(  
        user_id=user_id,  
        original_plan=original_plan,  
        nutrition_tip=nutrition_tip,  
    )
```

```
    db.add(plan)  
    db.commit()  
    db.refresh(plan)
```

```
    return plan
```

```
def get_user(  
    db: Session,  
    user_id: str  
) -> User | None:
```

```
    return db.scalar(  
        select(User).where(  
            User.user_id == user_id  
        )  
    )
```

```
def get_plan(  
    db: Session,  
    user_id: str  
) -> Plan | None:
```

```
    return db.scalar(  
        select(Plan).where(  
            Plan.user_id == user_id  
        )  
    )
```

```
    select(Plan)
    .where(Plan.user_id == user_id)
    .order_by(Plan.id.desc())
)
```

```
def update_plan(
    db: Session,
    plan: Plan,
    *,
    updated_plan: str,
    feedback: str,
    nutrition_tip: str,
) -> Plan:
```

```
    plan.updated_plan = updated_plan
    plan.feedback = feedback
    plan.nutrition_tip = nutrition_tip
```

```
    db.commit()
    db.refresh(plan)
```

```
    return plan
```

```
def get_all_users(
    db: Session
) -> list[User]:
```

```
    return list(
        db.scalars(
            select(User)
            .order_by(User.created_at.desc())
        ).all()
    )
```

```
def get_all_plans(
    db: Session
) -> list[Plan]:
```

```
    return list(
        db.scalars(
            select(Plan)
```

```
        .order_by(Plan.created_at.desc())
    ).all()
)
```

5. app/schemas.py

```
from typing import Literal
```

```
from pydantic import (
    BaseModel,
    Field,
    field_validator,
)
```

```
Goal = Literal[
    "weight loss",
    "muscle gain",
    "general wellness",
    "flexibility",
]
```

```
Intensity = Literal[
    "low",
    "medium",
    "high",
]
```

```
class UserInput(BaseModel):
```

```
    username: str = Field(
        min_length=2,
        max_length=120
    )
```

```
    user_id: str = Field(
        min_length=1,
        max_length=80
    )
```

```
    age: int = Field(
        ge=13,
        le=100
    )
```

```
weight: float = Field(
    gt=20,
    le=500
)
```

```
goal: Goal
```

```
intensity: Intensity
```

```
@field_validator(
    "username",
    "user_id"
)
```

```
@classmethod
def validate_text(
    cls,
    value: str
) -> str:
```

```
    value = value.strip()
```

```
    if not value:
        raise ValueError(
            "This field cannot be empty."
        )
```

```
    return value
```

```
class FeedbackRequest(BaseModel):
```

```
    user_id: str = Field(
        min_length=1,
        max_length=80
    )
```

```
    feedback: str = Field(
        min_length=3,
        max_length=1500
    )
```

```
class WorkoutExercise(BaseModel):
```

name: str

sets: int | None = None

reps: str | None = None

duration_minutes: int | None = None

rest_seconds: int | None = None

notes: str | None = None

class WorkoutDay(BaseModel):

day: str

focus: str

warm_up: str

exercises: list[WorkoutExercise]

cooldown: str

class WorkoutPlan(BaseModel):

title: str

overview: str

safety_note: str

days: list[WorkoutDay] = Field(
 min_length=7,
 max_length=7
)

class NutritionTip(BaseModel):

tip: str

rationale: str

hydration_note: str

```
class ErrorResponse(BaseModel):
```

```
    detail: str
```

```
6. app/gemini_generator.py
```

```
from google import genai
```

```
from google.genai import types
```

```
from .config import settings
```

```
from .schemas import (
```

```
    UserInput,
```

```
    WorkoutPlan,
```

```
)
```

```
def get_gemini_client() -> genai.Client:
```

```
    if not settings.gemini_api_key:
```

```
        raise RuntimeError(
```

```
            "GEMINI_API_KEY is not configured. "
```

```
            "Add it to your .env file."
```

```
        )
```

```
    return genai.Client(
```

```
        api_key=settings.gemini_api_key
```

```
    )
```

```
def generate_workout_gemini(
```

```
    user: UserInput
```

```
) -> WorkoutPlan:
```

```
    client = get_gemini_client()
```

```
    prompt = f"""
```

```
You are FitBuddy, an AI fitness planning assistant.
```

```
Create a personalized 7-day fitness plan.
```

```
User information:
```

Name: {user.username}
Age: {user.age}
Weight: {user.weight} kg
Fitness goal: {user.goal}
Workout intensity: {user.intensity}

Requirements:

1. Generate exactly 7 days.
2. Every day must contain:
day, focus, warm-up, exercises and cooldown.
3. Each exercise should include exercise name,
sets and repetitions OR duration.
4. Include rest period when appropriate.
5. Adapt the workout to the user's goal.
6. Adapt workload to the selected intensity.
7. Include appropriate recovery/rest.
8. Keep the plan practical.
9. Avoid dangerous or extreme recommendations.
10. Do not diagnose medical conditions.
11. Do not prescribe medication.
12. Include a safety note.
13. Tell the user to stop exercise if they experience
pain, dizziness, chest pain, breathing difficulty,
or unusual symptoms and seek appropriate professional help.

Return ONLY structured data matching the provided schema.

"""

```
response = client.models.generate_content(
    model=settings.gemini_workout_model,
    contents=prompt,
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=WorkoutPlan,
        temperature=0.4,
    ),
)

if not response.text:
    raise RuntimeError(
        "Gemini returned an empty response."
    )
```

```

        return WorkoutPlan.model_validate_json(
            response.text
        )
7. app/gemini_flash_generator.py
from google import genai
from google.genai import types

from .config import settings
from .schemas import (
    NutritionTip,
    UserInput,
)

def generate_nutrition_tip_with_flash(
    user: UserInput
) -> NutritionTip:

    if not settings.gemini_api_key:
        raise RuntimeError(
            "GEMINI_API_KEY is not configured."
        )

    client = genai.Client(
        api_key=settings.gemini_api_key
    )

    prompt = f"""
You are FitBuddy's nutrition and recovery assistant.

User:

Age: {user.age}
Weight: {user.weight} kg
Goal: {user.goal}
Workout intensity: {user.intensity}

Generate one concise practical nutrition
or recovery tip.

Focus on:

- balanced food choices

```

- protein where appropriate
- hydration
- recovery
- sleep
- healthy eating habits

Do NOT:

- diagnose medical conditions
- prescribe medication
- recommend extreme diets
- provide dangerous weight-loss instructions
- provide individualized medical treatment

Return only the structured schema.

"""

```

response = client.models.generate_content(
    model=settings.gemini_nutrition_model,
    contents=prompt,
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=NutritionTip,
        temperature=0.3,
    ),
)

```

```

if not response.text:
    raise RuntimeError(
        "Gemini returned an empty response."
    )

```

```

return NutritionTip.model_validate_json(
    response.text
)

```

```

8. app/updated_plan.py
from google import genai
from google.genai import types

```

```

from .config import settings
from .schemas import (
    UserInput,
    WorkoutPlan,
)

```

```
def update_workout_plan(
    original_plan: str,
    feedback: str,
    user: UserInput,
) -> WorkoutPlan:

    if not settings.gemini_api_key:
        raise RuntimeError(
            "GEMINI_API_KEY is not configured."
        )

    client = genai.Client(
        api_key=settings.gemini_api_key
    )

    prompt = f"""
You are FitBuddy's workout-plan revision assistant.
```

User information:

Age: {user.age}
 Weight: {user.weight} kg
 Goal: {user.goal}
 Intensity: {user.intensity}

ORIGINAL WORKOUT PLAN:

{original_plan}

USER FEEDBACK:

{feedback}

Create a revised 7-day workout plan.

Requirements:

1. Address the user's feedback.
2. Keep exactly 7 days.
3. Maintain the original fitness goal.
4. Maintain sensible progression.
5. Include sufficient recovery.

6. Do not diagnose medical conditions.
7. Do not prescribe medication.
8. Avoid unsafe recommendations.
9. Include a safety note.
10. Return ONLY the structured schema.

"""

```
response = client.models.generate_content(
    model=settings.gemini_workout_model,
    contents=prompt,
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=WorkoutPlan,
        temperature=0.4,
    ),
)
```

```
if not response.text:
    raise RuntimeError(
        "Gemini returned an empty response."
    )
```

```
return WorkoutPlan.model_validate_json(
    response.text
)
```

9. app/avlum.py

"""

Optional AVLUM integration.

The supplied FitBuddy project documentation does not specify:

- AVLUM API URL
- AVLUM SDK
- Authentication
- Endpoints
- Request schema
- Response schema

Therefore this file provides a generic integration boundary instead of inventing an AVLUM implementation.

"""

```
from dataclasses import dataclass
```

```

import httpx

from .config import settings

@dataclass
class AvlumEvent:

    event_type: str

    user_id: str

    payload: dict

async def emit_event(
    event: AvlumEvent
) -> bool:

    if not settings.avlum_enabled:
        return False

    if not settings.avlum_webhook_url:
        raise RuntimeError(
            "AVLUM is enabled but "
            "AVLUM_WEBHOOK_URL is not configured."
        )

    headers = {}

    if settings.avlum_api_key:
        headers["Authorization"] = (
            f"Bearer {settings.avlum_api_key}"
        )

    async with httpx.AsyncClient(
        timeout=10
    ) as client:

        response = await client.post(
            settings.avlum_webhook_url,
            json={
                "event_type": event.event_type,
                "user_id": event.user_id,

```

```
        "payload": event.payload,
    },
    headers=headers,
)
```

```
response.raise_for_status()
```

```
return True
```

10. app/routes.py

```
import json
```

```
from fastapi import (
    APIRouter,
    Depends,
    Form,
    HTTPException,
    Request,
    status,
)
```

```
from fastapi.responses import (
    HTMLResponse,
    RedirectResponse,
)
```

```
from fastapi.templating import Jinja2Templates
```

```
from sqlalchemy.orm import Session
```

```
from .avlum import (
    AvlumEvent,
    emit_event,
)
```

```
from .config import settings
```

```
from .database import (
    get_all_plans,
    get_all_users,
    get_db,
    get_plan,
    get_user,
    save_plan,
    save_user,
```

```

        update_plan,
    )

    from .gemini_flash_generator import (
        generate_nutrition_tip_with_flash,
    )

    from .gemini_generator import (
        generate_workout_gemini,
    )

    from .schemas import (
        FeedbackRequest,
        UserInput,
    )

    from .updated_plan import (
        update_workout_plan,
    )

    router = APIRouter()

    templates = Jinja2Templates(
        directory="templates"
    )

    def error_page(
        request: Request,
        message: str
    ):

        return templates.TemplateResponse(
            request=request,
            name="error.html",
            context={
                "request": request,
                "message": message,
            },
            status_code=500,
        )

```

```

def is_admin(
    request: Request
) -> bool:

    return bool(
        request.session.get(
            "admin_authenticated"
        )
    )

@router.get(
    "/",
    response_class=HTMLResponse
)
def home(request: Request):

    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={
            "request": request
        },
    )

@router.post(
    "/generate-workout",
    response_class=HTMLResponse
)
async def generate_workout(
    request: Request,
    username: str = Form(...),
    user_id: str = Form(...),
    age: int = Form(...),
    weight: float = Form(...),
    goal: str = Form(...),
    intensity: str = Form(...),
    db: Session = Depends(get_db),
):

    try:

        user = UserInput(

```

```

        username=username,
        user_id=user_id,
        age=age,
        weight=weight,
        goal=goal,
        intensity=intensity,
    )

    workout = generate_workout_gemini(
        user
    )

    nutrition = (
        generate_nutrition_tip_with_flash(
            user
        )
    )

    save_user(
        db,
        user_id=user.user_id,
        username=user.username,
        age=user.age,
        weight=user.weight,
        goal=user.goal,
        intensity=user.intensity,
    )

    save_plan(
        db,
        user_id=user.user_id,
        original_plan=workout.model_dump_json(
            indent=2
        ),
        nutrition_tip=nutrition.model_dump_json(
            indent=2
        ),
    )

    await emit_event(
        AvlumEvent(
            event_type="plan.created",
            user_id=user.user_id,
            payload={

```

```

        "goal": user.goal,
        "intensity": user.intensity,
    },
)
)

return templates.TemplateResponse(
    request=request,
    name="result.html",
    context={
        "request": request,
        "user": user,
        "workout": workout.model_dump(),
        "nutrition": nutrition.model_dump(),
        "is_updated": False,
        "message": (
            "Your personalized "
            "7-day plan is ready."
        ),
    },
)

```

except Exception as exc:

```

    return error_page(
        request,
        str(exc)
    )

```

```

@router.post(
    "/submit-feedback",
    response_class=HTMLResponse
)
async def submit_feedback(
    request: Request,
    user_id: str = Form(...),
    feedback: str = Form(...),
    db: Session = Depends(get_db),
):

```

try:

```

    feedback_request = FeedbackRequest(

```

```

        user_id=user_id,
        feedback=feedback,
    )

    user_record = get_user(
        db,
        feedback_request.user_id
    )

    plan_record = get_plan(
        db,
        feedback_request.user_id
    )

    if not user_record:
        raise HTTPException(
            status_code=404,
            detail="User not found."
        )

    if not plan_record:
        raise HTTPException(
            status_code=404,
            detail="Workout plan not found."
        )

    user = UserInput(
        username=user_record.username,
        user_id=user_record.user_id,
        age=user_record.age,
        weight=user_record.weight,
        goal=user_record.goal,
        intensity=user_record.intensity,
    )

    revised = update_workout_plan(
        original_plan=plan_record.original_plan,
        feedback=feedback_request.feedback,
        user=user,
    )

    nutrition = (
        generate_nutrition_tip_with_flash(
            user

```

```

    )
)

update_plan(
    db,
    plan_record,
    updated_plan=revised.model_dump_json(
        indent=2
    ),
    feedback=feedback_request.feedback,
    nutrition_tip=nutrition.model_dump_json(
        indent=2
    ),
)

```

```

await emit_event(
    AvlumEvent(
        event_type="plan.updated",
        user_id=user.user_id,
        payload={
            "feedback":
                feedback_request.feedback
        },
    )
)

```

```

return templates.TemplateResponse(
    request=request,
    name="result.html",
    context={
        "request": request,
        "user": user,
        "workout": revised.model_dump(),
        "nutrition": nutrition.model_dump(),
        "is_updated": True,
        "message": (
            "Your plan was updated "
            "using your feedback."
        ),
    },
)

```

```

except HTTPException:
    raise

```

except Exception as exc:

```
    return error_page(
        request,
        str(exc)
    )
```

```
@router.get(
    "/api/users/{user_id}"
)
```

```
def api_user(
    user_id: str,
    db: Session = Depends(get_db),
):
```

```
    user = get_user(
        db,
        user_id
    )
```

```
    plan = get_plan(
        db,
        user_id
    )
```

if not user or not plan:

```
    raise HTTPException(
        status_code=404,
        detail="User or plan not found."
    )
```

```
return {
    "user": {
        "user_id": user.user_id,
        "username": user.username,
        "age": user.age,
        "weight": user.weight,
        "goal": user.goal,
        "intensity": user.intensity,
    },
    "plan": {
```

```

        "original_plan":
            json.loads(
                plan.original_plan
            ),

        "updated_plan":
            json.loads(
                plan.updated_plan
            )
            if plan.updated_plan
            else None,

        "nutrition_tip":
            json.loads(
                plan.nutrition_tip
            ),

        "feedback":
            plan.feedback,
    },
}

```

```

@router.get("/health")
def health():

```

```

    return {
        "status": "ok",
        "service": "fitbuddy",
    }

```

```

@router.get(
    "/admin/login",
    response_class=HTMLResponse
)

```

```

def admin_login(
    request: Request
):

```

```

    return templates.TemplateResponse(
        request=request,
        name="admin_login.html",
    )

```

```

@router.post("/admin/login")
def admin_login_submit(
    request: Request,
    password: str = Form(...),
):

    if password != settings.admin_password:

        return templates.TemplateResponse(
            request=request,
            name="admin_login.html",
            context={
                "error":
                    "Invalid admin password."
            },
            status_code=401,
        )

    request.session[
        "admin_authenticated"
    ] = True

    return RedirectResponse(
        "/view-all-users",
        status_code=303,
    )


@router.post("/admin/logout")
def admin_logout(
    request: Request
):

    request.session.clear()

    return RedirectResponse(
        "/admin/login",
        status_code=303,
    )


@router.get(

```

```

        "/view-all-users",
        response_class=HTMLResponse
    )
def view_all_users(
    request: Request,
    db: Session = Depends(get_db),
):
    if not is_admin(request):
        return RedirectResponse(
            "/admin/login",
            status_code=303,
        )

    users = get_all_users(db)

    plans = {
        plan.user_id: plan
        for plan in get_all_plans(db)
    }

    return templates.TemplateResponse(
        request=request,
        name="all_users.html",
        context={
            "request": request,
            "users": users,
            "plans": plans,
        },
    )
{% extends "base.html" %}

{% block title %}
FitBuddy | Your Plan
{% endblock %}

{% block content %}

<section class="hero compact">

    <p class="eyebrow">

```

```
{% if is_updated %}
  UPDATED PLAN
{% else %}
  YOUR PLAN
{% endif %}
```

</p>

```
<h1>
  {{ message }}
</h1>
```

```
<p>
  Hello {{ user.username }}.
```

```
  Goal:
  {{ user.goal }}
```

|

```
  Intensity:
  {{ user.intensity }}
</p>
```

</section>

```
<section class="card">
```

```
<h2>
  {{ workout.title }}
</h2>
```

```
<p>
  {{ workout.overview }}
</p>
```

</section>

```
<section class="plan-grid">
```

```
{% for day in workout.days %}
```

```
<article class="day-card">
```

```
<h3>
```

```
  {{ day.day }}
```

```
</h3>
```

```
<p>
```

```
  <strong>Focus:</strong>
```

```
  {{ day.focus }}
```

```
</p>
```

```
<p>
```

```
  <strong>Warm-up:</strong>
```

```
  {{ day.warm_up }}
```

```
</p>
```

```
<h4>
```

```
  Exercises
```

```
</h4>
```

```
<ul>
```

```
{% for exercise in day.exercises %}
```

```
<li>
```

```
  <strong>
```

```
    {{ exercise.name }}
```

```
</strong>
```

```
{% if exercise.sets %}
```

```
  — {{ exercise.sets }} sets
```

```
{% endif %}
```

```
{% if exercise.reps %}
```

```
  × {{ exercise.reps }}
```

```
{% endif %}
```

```
{% if exercise.duration_minutes %}
```

```
  — {{ exercise.duration_minutes }} min
```

```
{% endif %}
```

```
{% if exercise.rest_seconds %}  
  — Rest {{ exercise.rest_seconds }} sec  
{% endif %}
```

```
{% if exercise.notes %}
```

```
  <br>  
  {{ exercise.notes }}
```

```
{% endif %}
```

```
</li>
```

```
{% endfor %}
```

```
</ul>
```

```
<p>
```

```
  <strong>  
    Cooldown:  
  </strong>
```

```
  {{ day.cooldown }}
```

```
</p>
```

```
</article>
```

```
{% endfor %}
```

```
</section>
```

```
<section class="card">
```

```
  <h2>  
    Nutrition & Recovery  
  </h2>
```

```
<p>
```

```
    <strong>Tip:</strong>
    {{ nutrition.tip }}
</p>
```

```
<p>
    <strong>Why:</strong>
    {{ nutrition.rationale }}
</p>
```

```
<p>
    <strong>Hydration:</strong>
    {{ nutrition.hydration_note }}
</p>
```

```
</section>
```

```
<section class="card safety">
```

```
    <h2>
        Safety Note
    </h2>
```

```
    <p>
        {{ workout.safety_note }}
    </p>
```

```
</section>
```

```
<section class="card">
```

```
    <h2>
        Improve Your Plan
    </h2>
```

```
    <p>
        Tell FitBuddy what you want to change.
    </p>
```

```
<form
    method="post"
    action="/submit-feedback"
```

>

```
<input
  type="hidden"
  name="user_id"
  value="{{ user.user_id }}"
>
```

<label>

Feedback

```
<textarea
  name="feedback"
  rows="5"
  maxlength="1500"
  required
  placeholder="Example: The workouts are too difficult. Please reduce intensity."
></textarea>
```

</label>

```
<button
  class="button"
  type="submit"
>
  Update Plan
</button>
```

</form>

</section>

```
{% endblock %}
{% extends "base.html" %}
```

```
{% block title %}
FitBuddy | Create Your Plan
{% endblock %}
```

```
{% block content %}
```

```
<section class="hero">
```

```
<div>
```

```
<p class="eyebrow">  
  AI FITNESS PLANNER  
</p>
```

```
<h1>  
  Create your personalized workout plan.  
</h1>
```

```
<p>  
  Enter your details and FitBuddy will  
  generate a practical 7-day workout plan  
  with nutrition and recovery guidance.  
</p>
```

```
</div>
```

```
</section>
```

```
<section class="card">
```

```
<h2>  
  Your Details  
</h2>
```

```
<form  
  method="post"  
  action="/generate-workout"  
  class="form-grid"  
>
```

```
<label>  
  Name
```

```
<input  
  type="text"  
  name="username"  
  required
```

```
        minlength="2"
    >
</label>
```

```
<label>
    User ID

    <input
        type="text"
        name="user_id"
        required
    >
</label>
```

```
<label>
    Age

    <input
        type="number"
        name="age"
        min="13"
        max="100"
        required
    >
</label>
```

```
<label>
    Weight (kg)

    <input
        type="number"
        name="weight"
        min="20"
        max="500"
        step="0.1"
        required
    >
</label>
```

```
<label>
```

Goal

```
<select
  name="goal"
  required
>

  <option value="weight loss">
    Weight Loss
  </option>

  <option value="muscle gain">
    Muscle Gain
  </option>

  <option value="general wellness">
    General Wellness
  </option>

  <option value="flexibility">
    Flexibility
  </option>

</select>

</label>
```

```
<label>
  Intensity

  <select
    name="intensity"
    required
  >

    <option value="low">
      Low
    </option>

    <option
      value="medium"
      selected
    >
```

```
        Medium
    </option>
```

```
    <option value="high">
        High
    </option>
```

```
</select>
```

```
</label>
```

```
<div class="full">
```

```
    <button
        class="button"
        type="submit"
    >
        Generate My Plan
    </button>
```

```
</div>
```

```
</form>
```

```
</section>
```

```
{% endblock %}
```

```
11. app/main.py
```

```
from fastapi import FastAPI
```

```
from fastapi.staticfiles import StaticFiles
```

```
from starlette.middleware.sessions import SessionMiddleware
```

```
from .config import settings
```

```
from .database import init_db
```

```
from .routes import router
```

```
app = FastAPI(
    title="FitBuddy – AI Fitness Plan Generator",
    description=(
        "AI-assisted fitness planning using "
        "Gemini, FastAPI, SQLite and Jinja2."
    ),
)
```

```
    version="1.0.0",  
)
```

```
app.add_middleware(  
    SessionMiddleware,  
    secret_key=settings.session_secret,  
    same_site="lax",  
    https_only=False,  
)
```

```
app.mount(  
    "/static",  
    StaticFiles(directory="static"),  
    name="static",  
)
```

```
app.include_router(router)
```

```
@app.on_event("startup")
```

```
def startup():
```

```
    init_db()
```

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <meta
```

```
        name="viewport"
```

```
        content="width=device-width, initial-scale=1.0"
```

```
    >
```

```
<title>
```

```
    {% block title %}
```

```
        FitBuddy
```

```
    {% endblock %}
```

```
</title>
```

```
<link
```

```
    rel="stylesheet"
```

```
        href="{{ url_for('static', path='css/styles.css') }}"
    >
</head>
```

```
<body>
```

```
<header class="topbar">
```

```
    <div class="container nav">
```

```
        <a class="brand" href="/">
            FitBuddy
        </a>
```

```
        <nav>
            <a href="/">
                Planner
            </a>
```

```
            <a href="/admin/login">
                Admin
            </a>
```

```
            <a href="/docs">
                API Docs
            </a>
        </nav>
```

```
    </div>
```

```
</header>
```

```
<main class="container">
```

```
    {% block content %}
    {% endblock %}
```

```
</main>
```

```
<footer class="footer">
```

```
    <div class="container">
```

```
FitBuddy – AI Fitness Plan Generator
</div>
```

```
</footer>
```

```
</body>
```

```
</html>
```

```
{% extends "base.html" %}
```

```
{% block title %}
```

```
FitBuddy | All Users
```

```
{% endblock %}
```

```
{% block content %}
```

```
<section class="hero compact">
```

```
    <p class="eyebrow">
```

```
        ADMIN DASHBOARD
```

```
    </p>
```

```
    <h1>
```

```
        All Users
```

```
    </h1>
```

```
    <p>
```

```
        View registered users and their
        latest plan information.
```

```
    </p>
```

```
</section>
```

```
<section class="card table-wrap">
```

```
    <table>
```

```
        <thead>
```

```
            <tr>
```

```
                <th>User ID</th>
```

```
                <th>Name</th>
```

```
                <th>Age</th>
```

```

        <th>Weight</th>
        <th>Goal</th>
        <th>Intensity</th>
        <th>Feedback</th>
    </tr>

</thead>

<tbody>

{% for user in users %}

    {% set plan = plans.get(user.user_id) %}

    <tr>

        <td>
            {{ user.user_id }}
        </td>

        <td>
            {{ user.username }}
        </td>

        <td>
            {{ user.age }}
        </td>

        <td>
            {{ user.weight }} kg
        </td>

        <td>
            {{ user.goal }}
        </td>

        <td>
            {{ user.intensity }}
        </td>

        <td>

            {% if plan and plan.feedback %}

```

```

        {{ plan.feedback }}

    {% else %}

        No feedback

    {% endif %}

</td>

</tr>

{% else %}

<tr>

    <td colspan="7">
        No users yet.
    </td>

</tr>

{% endfor %}

</tbody>

</table>

</section>

<form
    method="post"
    action="/admin/logout"
>

    <button
        class="button secondary"
        type="submit"
    >
        Logout
    </button>

```

```
</form>
```

```
{% endblock %}
```

```
12. templates/base.html
```

```
HTML
```

```
13. templates/index.html
```

```
HTML
```

```
{% extends "base.html" %}
```

```
{% block title %}
```

```
FitBuddy | Admin Login
```

```
{% endblock %}
```

```
{% block content %}
```

```
<section class="card narrow">
```

```
<h1>
```

```
Admin Login
```

```
</h1>
```

```
{% if error %}
```

```
<div class="alert">
```

```
  {{ error }}
```

```
</div>
```

```
{% endif %}
```

```
<form
```

```
  method="post"
```

```
  action="/admin/login"
```

```
>
```

```
<label>
```

```
  Password
```

```
<input
```

```
  type="password"
```

```
  name="password"
```

```

        required
    >

</label>

<button
    class="button"
    type="submit"
>
    Login
</button>

</form>

</section>

{% endblock %}
14. templates/result.html
HTML
15. templates/admin_login.html
HTML
16. templates/all_users.html
HTML
{% extends "base.html" %}

{% block title %}
FitBuddy | Error
{% endblock %}

{% block content %}

<section class="card narrow">

    <h1>
        Something went wrong
    </h1>

    <p>
        {{ message }}
    </p>

    <a

```

```
        class="button"
        href="/"
    >
        Back to FitBuddy
    </a>
```

```
</section>
```

```
{% endblock %}
```

```
17. templates/error.html
```

```
HTML
```

```
18. static/css/styles.css
```

```
* {
    box-sizing: border-box;
}
```

```
body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #07140f;
    color: #eef8f2;
    line-height: 1.6;
}
```

```
a {
    color: inherit;
    text-decoration: none;
}
```

```
.container {
    width: min(1120px, 92%);
    margin: auto;
}
```

```
.topbar {
    border-bottom: 1px solid #244b3a;
    background: #091c15;
}
```

```
.nav {
    min-height: 70px;
    display: flex;
    align-items: center;
    justify-content: space-between;
```

```
    gap: 20px;  
}
```

```
.brand {  
  font-size: 1.5rem;  
  font-weight: 800;  
}
```

```
nav {  
  display: flex;  
  gap: 20px;  
}
```

```
nav a:hover {  
  text-decoration: underline;  
}
```

```
.hero {  
  padding: 70px 0 40px;  
}
```

```
.hero.compact {  
  padding-bottom: 25px;  
}
```

```
.hero h1 {  
  font-size: clamp(2rem, 5vw, 4rem);  
  line-height: 1.05;  
  margin: 8px 0 18px;  
}
```

```
.eyebrow {  
  letter-spacing: .15em;  
  font-size: .8rem;  
  font-weight: 700;  
  color: #8fe0b2;  
}
```

```
.card,  
.day-card {  
  background: #0d251b;  
  border: 1px solid #24513d;  
  border-radius: 18px;  
  padding: 24px;
```

```
    margin: 20px 0;
}
```

```
.form-grid {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 18px;
}
```

```
.full {
    grid-column: 1 / -1;
}
```

```
label {
    display: block;
    font-weight: 700;
}
```

```
input,
select,
textarea {
    width: 100%;
    margin-top: 8px;
    border: 1px solid #37654f;
    border-radius: 10px;
    padding: 12px;
    background: #07140f;
    color: #eef8f2;
    font: inherit;
}
```

```
textarea {
    resize: vertical;
}
```

```
.button {
    display: inline-block;
    border: 0;
    border-radius: 10px;
    padding: 12px 20px;
    background: #64d994;
    color: #06110b;
    font-weight: 800;
    cursor: pointer;
```

```
    margin-top: 14px;  
}
```

```
.button:hover {  
    opacity: .9;  
}
```

```
.button.secondary {  
    background: #294c3b;  
    color: #eef8f2;  
}
```

```
.plan-grid {  
    display: grid;  
    grid-template-columns: repeat(2, 1fr);  
    gap: 20px;  
}
```

```
.day-card {  
    margin: 0;  
}
```

```
.day-card h3 {  
    margin-top: 0;  
}
```

```
.day-card li {  
    margin: 10px 0;  
}
```

```
.safety {  
    border-color: #8c7440;  
}
```

```
.narrow {  
    max-width: 600px;  
    margin: 60px auto;  
}
```

```
.alert {  
    padding: 12px;  
    border-radius: 10px;  
    background: #542727;  
    margin: 12px 0;
```

```

}

.table-wrap {
  overflow-x: auto;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th,
td {
  text-align: left;
  padding: 12px;
  border-bottom: 1px solid #2b513f;
}

.footer {
  padding: 40px 0;
  margin-top: 60px;
  border-top: 1px solid #244b3a;
  color: #a8c4b4;
}

@media (max-width: 760px) {

  .form-grid,
  .plan-grid {
    grid-template-columns: 1fr;
  }

  .full {
    grid-column: auto;
  }

  .nav {
    align-items: flex-start;
    padding: 15px 0;
    flex-direction: column;
  }

  nav {
    flex-wrap: wrap;
  }

```

```

    }
}
19. tests/test_app.py
from fastapi.testclient import TestClient

from app.main import app

client = TestClient(app)

def test_health():

    response = client.get(
        "/health"
    )

    assert response.status_code == 200

    assert response.json()["status"] == "ok"

def test_home():

    response = client.get("/")

    assert response.status_code == 200

    assert "FitBuddy" in response.text

def test_admin_requires_login():

    response = client.get(
        "/view-all-users",
        follow_redirects=False
    )

    assert response.status_code == 303

    assert (
        response.headers["location"]
        == "/admin/login"
    )

```

```
20. .env.example
GEMINI_API_KEY=your_real_gemini_api_key

GEMINI_WORKOUT_MODEL=gemini-3.8-flash

GEMINI_NUTRITION_MODEL=gemini-3.8-flash

DATABASE_URL=sqlite:///./fitbuddy.db

ADMIN_PASSWORD=change-me

SESSION_SECRET=replace-with-a-long-random-secret

AVLUM_ENABLED=false

AVLUM_WEBHOOK_URL=
```

```
AVLUM_API_KEY=
21. requirements.txt
fastapi>=0.115,<1
uvicorn[standard]>=0.34,<1
jinja2>=3.1,<4
sqlalchemy>=2.0,<3
python-multipart>=0.0.20,<1
pydantic>=2.10,<3
pydantic-settings>=2.7,<3
google-genai>=1.40,<2
httpx>=0.28,<1
itsdangerous>=2.2,<3
pytest>=8.3,<9
```

```
22. .vscode/settings.json
{
  "python.testing.pytestEnabled": true,
  "python.testing.unittestEnabled": false,
  "python.testing.pytestArgs": [
    "tests"
  ],
  "python.analysis.typeCheckingMode": "basic"
}
```

```
23. .vscode/launch.json
{
  "version": "0.2.0",
  "configurations": [
    {
```

```

        "name": "FitBuddy FastAPI",
        "type": "debugpy",
        "request": "launch",
        "module": "uvicorn",
        "args": [
            "app.main:app",
            "--reload"
        ],
        "jinja": true
    }
]
}

```

24. .gitignore

```

.venv/
venv/
__pycache__/
*.py[cod]
.pytest_cache/
.env
fitbuddy.db
.vscode/
.DS_Store

```

25. README.md

FitBuddy – AI Fitness Plan Generator

Project

FitBuddy is an AI-assisted fitness plan generator built with:

- FastAPI
- Jinja2
- SQLite
- SQLAlchemy
- Gemini AI
- Optional AVLUM webhook integration

Setup

Create a virtual environment:

```

```bash
python -m venv .venv
Activate it.
Windows PowerShell:

```

`.venv\Scripts\Activate.ps1`

Windows CMD:

`.venv\Scripts\activate`

Install dependencies:

`pip install -r requirements.txt`

Create `.env` from `.env.example`.

Add your Gemini API key:

`GEMINI_API_KEY=your_api_key`

Start the application:

`uvicorn app.main:app --reload`

Open:

`http://127.0.0.1:8000`

Useful URLs

Home:

`/`

API documentation:

`/docs`

Health:

`/health`

Admin login:

`/admin/login`

Admin dashboard:

`/view-all-users`

Tests

`pytest -q`

Workflow

User enters profile information.

FitBuddy generates a 7-day workout plan.

Gemini generates nutrition/recovery guidance.

User data is stored in SQLite.

User can submit feedback.

Gemini creates a revised plan.

Optional AVLUM events can be emitted.

AVLUM

The supplied project documentation does not provide a concrete AVLUM API contract.

Therefore `app/avlum.py` contains a generic webhook integration boundary.

Actual AVLUM endpoint, authentication and request format should be added when its API documentation is available.

**\*\*Important:\*\*** ``GEMINI_WORKOUT_MODEL`` and ``GEMINI_NUTRITION_MODEL`` values may need to be changed to a Gemini model actually available in your API account.